République de Tunisie

Ministère de l'enseignement supérieur & de la recherche scientifique

Université de Carthage

SUP'COM

**École supérieure de communication de Tunis (SUP'COM)**

# Rapport de stage d'ingénieur

## Construction des classificateurs des cibles radar FMCW 24 GHZ pour la surveillance intelligente du trafic à l'aide des modèles deep learning.

Du 20 juillet au 31 août 2020

Réalisé par : **Amine DAGDOUG**

Étudiant en 2ème année d'ingénierie

Supervisé par : **Pr. Adel GHAZEL**

Professeur en télécommunications à SUP'COM

**Ing. Kawthar OSMAN**

Ingénieur système embarqué à WEVIOO

Entreprise hôte : Wevioo Tunisie

wevioo
INSPIRED SOLUTIONS

Année universitaire: 2019/2020

# Remerciements

La mise en œuvre de ce projet n'aurait pas été possible si je n'avais pas eu le soutien de nombreuses personnes. C'est pourquoi je tiens à leur exprimer ma sincère gratitude.

Tout d'abord, je voudrais exprimer ma sincère et profonde gratitude à mon superviseur académique, M. Adel Ghazel, professeur à SUP'COM, qui m'a donné l'opportunité d'être un stagiaire au sein de groupe WEVIOO et pour ses conseils, ses précieuses suggestions et son assistance constante tout au long de ce stage. En second lieu, je remercie Mme. Kawthar Osman, membre de l'équipe du groupe Wevioo, pour sa collaboration fructueuse et sa supervision pendant mon stage.

Néanmoins, j'exprime ma gratitude envers les ingénieurs de l'équipe Wevioo pour leur aide et leur aimable coopération, qui m'aident à mener à bien ce projet.

En dernier lieu je remercie l'administration de SUP'COM qu'elle nous a permis de s'installer dans l'une de ces salles pour réaliser ce projet-là dans des conditions favorables.

# Résumé et mots-clés

Le trafic routier a un effet important sur les activités métropolitaines, en particulier aux heures de pointe, lorsqu'il a des répercussions sur des domaines tels que l'économie et l'environnement. Au cours des dernières années, les nouvelles technologies dans les domaines de l'information et de la communication ont conduit à l'introduction de systèmes de transport intelligents.

L'objectif de notre projet, basé sur les exigences de l'application de l'entreprise hôte, est de développer et mettre en œuvre une solution algorithmique pour la classification des cibles du radar FMCW à 24 GHz. Dans le cadre de cet objectif, les activités du projet ont commencé par l'étude de techniques basées sur l'intelligence artificielle, principalement utilisées pour la classification des cibles mobiles du trafic routier, y compris les camions, les véhicules, les motos et les piétons. En considérant à la fois les 2 approches d'apprentissage profond (DL) : ANN et CNN, nous avons généré nos données à l'aide de la bibliothèque Simulink de Matlab qui simule le fonctionnement de kit radar 24 GHz DemoRad de Analog Devices, puis nous avons procédé au prétraitement des données pour construire la base de données de formation et de test. Différents modèles de formation ont été développés pour les techniques DL et leurs performances ont été testées sur des scénarios de trafic différents. Une précision de classification de 96% a été obtenue avec le modèle ANN et 98% avec le modèle CNN.

Mots clés : Radar FMCW, surveillance du trafic routier, classification des cibles radar, apprentissage approfondi (deep learning).

# Table des matières

**Remerciements** ..... 2
**Résumé et mots-clés** ..... 3
**Tables des figures** ..... 6
**Liste des acronymes :** ..... 7
**Introduction générale :** ..... 8
**Chapitre 1 : Cadre du projet de stage** ..... 10
**1.1 Introduction** ..... 10
**1.2 Aperçu du projet de stage** ..... 10
1.2.1 Reconnaissance des cibles pour les applications de surveillance du trafic ..... 10
1.2.2 Techniques de reconnaissance des cibles ..... 11
**1.3 Objectifs du projet et méthodologie de conception** ..... 11
1.3.1 Objectifs du projet ..... 11
1.3.2 Planification du projet ..... 12
**1.4 Environnement de travail du projet de stage** ..... 12
1.4.1 Présentation de l'entreprise d'accueil ..... 12
1.4.2 Outils matériels et logiciels ..... 13
**1.5 Conclusion** ..... 15
**Chapitre 2 : Aspects de la conception de la reconnaissance des cibles radar** ..... 16
**2.1 Introduction** ..... 16
**2.2 Radar FMCW 24GHZ** ..... 16
**2.3 Réseau de neurones** ..... 17
**2.4 Réseau neuronal convolutif** ..... 18
**2.5 Conclusion** ..... 20
**Chapitre 3 : Exploration des données** ..... 21
**3.1 Introduction** ..... 21
**3.2 Présentation de l'ensemble des données** ..... 21
**3.3 Nettoyage des données en double** ..... 21
**3.4 Augmentation de la base des données** ..... 22
**3.5 Visualisation des données** ..... 23
**3.6 Normalisation des données** ..... 24

**3.7 encodage des labels** ..... 24
**3.8 Conclusion** ..... 24
**Chapitre 4 : Description des modèles** ..... 25
**4.1 Introduction** ..... 25
**4.2 Développement du modèle basé sur l'ANN** ..... 25
**4.3 Développement du modèle basé sur le CNN** ..... 29
**4.3 Conclusion** ..... 33
**Conclusion et perspectives** ..... 34

# Tables des figures

Figure 1 . 1: Exemple de reconnaissance de cibles radar pour les applications de surveillance du trafic . 11
Figure 1 . 2: Logo WEVIOO ..... 12
Figure 1 . 3: Présence de WEVIOO dans le monde ..... 13
Figure 1 . 4: Google Colab ..... 13
Figure 1 . 5: Matlab & Simulink ..... 14
Figure 1 . 6: Python ..... 14

Figure 2 . 1: Composants de l'émetteur-récepteur radar DemoRad ..... 16
Figure 2 . 2: Architecture des étapes de traitement de la détection des cibles ..... 17
Figure 2 . 3: Vue simplifiée d'un réseau de neurones artificiels ..... 18
Figure 2 . 4: Exemple d'un réseau neuronal convolutif en 2D ..... 19
Figure 2 . 5: Matrice d'entrée multipliée par le kernel ou la matrice de filtrage ..... 19
Figure 2 . 6: Couche de Max polling 2*2 ..... 20
Figure 2 . 7: Couche entièrement connectée dans un CNN ..... 20

Figure 3 . 1: Classes des objets détectés et classifiés ..... 21
Figure 3 . 2: Nettoyage des données d'apprentissage ..... 22
Figure 3 . 3: Augmentation de la base des données ..... 22
Figure 3 . 4: Représentation des proportions de l'ensemble des données ..... 23
Figure 3 . 5: Exemple d'une DSP d'un piéton ..... 23
Figure 3 . 6: Exemple d'une DSP d'une moyenne voiture ..... 23
Figure 3 . 7: Effet de la normalisation des données ..... 24
Figure 3 . 8: Encodage des labels ..... 24

Figure 4 . 1: Modèle ANN ..... 25
Figure 4 . 2: Compilation de modèle ANN ..... 26
Figure 4 . 3: Entrainment de modèle ANN ..... 26
Figure 4 . 4: Phase d'entrainement ..... 27
Figure 4 . 5: Courbes de précision ..... 28
Figure 4 . 6: Courbes de perte ..... 28
Figure 4 . 7: Matrice de confusion de modèle ANN ..... 28
Figure 4 . 8: Modèle CNN ..... 30
Figure 4 . 9: Compilation de modèle CNN ..... 31
Figure 4 . 10: Entrainement de modèle CNN ..... 31
Figure 4 . 11: Phase d'entrainement ..... 32
Figure 4 . 12: Courbes de précision ..... 32
Figure 4 . 13: Courbes de perte ..... 32
Figure 4 . 14: Matrice de confusion de modèle CNN ..... 33

# Liste des acronymes :

AI: Artificial Intelligence.

ANN : Artificial Neural Networks.

AR: Accuracy Rate.

CNN : Convolutional Neural Networks.

CM: Confusion Matrix.

DL : Deep Learning.

DNN : Deep Neural Networks.

FFT : Fast Fourier transform.

FMCW : Frequency-Modulated Continuous-Wave.

LIDAR : light detection and ranging.

RD : Range Doppler.

2D-CNN : 2 Dimensional Convolutional Neural Networks.

# Introduction générale :

Le traitement automatique des informations en temps réel sur le trafic est une caractéristique importante pour analyser le trafic. Le nombre de capteurs de trafic augmente rapidement, ainsi que la quantité d'informations utiles disponibles pour les applications de surveillance du trafic. Comme nous nous dirigeons vers des villes intelligentes, la surveillance du trafic devient un sujet important à aborder afin d'améliorer la sécurité de la route et de maintenir une bonne supervision.

Dans le scénario actuel, la plupart des techniques de surveillance basées sur la vision nécessitent des données de trafic provenant d'un vaste réseau sensoriel, ce qui est extrêmement coûteux. Par conséquent, un nouveau système de surveillance du trafic basé sur l'analyse et l'évaluation en temps réel des situations de trafic dans pratiquement toutes les conditions météorologiques est nécessaire, dont une partie importante est la détection et la classification des cibles mobiles.

D'autre part, les capteurs radar offrent en général la possibilité de mesurer avec une extrême précision la distance de la cible, la vitesse radiale et l'angle d'azimut pour tous les objets se trouvant dans la zone d'observation. Ces paramètres peuvent être mesurés simultanément, même dans des situations de cibles multiples, ce qui est largement utilisé avec les nouvelles technologies du radar FMCW, mais le développement de solutions de classification des cibles basées sur l'intelligence artificielle est encore en phase de prototypage dans les laboratoires de recherche et aucun produit mature n'est disponible sur le marché.

Compte tenu du potentiel économique élevé des capteurs intelligents de faible puissance basés sur les radars FMCW pour les applications de contrôle et de surveillance du trafic, le groupe WEVIOO est engagé au sein de sociétés leaders mondiales dans des activités de recherche et développement liées à ce sujet. C'est dans cette perspective, l'objectif principal de mon projet est de trouver une solution intelligente pour la classification des objectifs de trafic routier en zone urbaine en se basant sur des algorithmes deep learning.

Ce rapport est divisé en ces différents chapitres suivants :

Le premier chapitre donne un bref aperçu de l'entreprise d'accueil, du contexte du projet et de ses objectifs, ainsi qu'un aperçu de l'organisation et de la structure du projet, la définition du cadre et des outils de développement utilisés.

Dans le deuxième chapitre, nous présenterons une étude sur les techniques basées sur deep learning pour la classification des cibles radar, la méthodologie de conception pour développer une solution basée sur l'IA pour la reconnaissance des cibles radar.

Dans le troisième chapitre, nous expliquerons en détail le traitement depuis la géneration et le nettoyage des données jusqu'à l'obtention des ensembles de données DL.

Dans le quatrième chapitre, nous présenterons la conception de la solution proposée, puis nous développerons les modèles DL avec les outils appropriés et nous l'évaluerons à la fin.

Enfin, nous concluons par une conclusion générale qui présente un résumé du travail effectué et ouvre quelques perspectives.

# Chapitre 1 : Cadre du projet de stage

## 1.1 Introduction

Dans ce chapitre, nous souhaitons donner un aperçu global du cadre du projet de stage. Tout d'abord, un aperçu du projet va être présenté, puis les objectifs du projet et la méthodologie de conception vont être introduits. A la fin, une présentation de l'environnement de travail du projet de stage aura lieu.

## 1.2 Aperçu du projet de stage

### 1.2.1 Reconnaissance des cibles pour les applications de surveillance du trafic

Les systèmes de transport intelligents (STI) participent pleinement à la surveillance du trafic routier grâce aux moyens et équipements dynamiques utilisés par le gouvernement sur les réseaux routiers et autoroutiers. Ils constituent également un atout majeur pour l'information des automobilistes et plus généralement pour l'exploitation de la route.

La surveillance du trafic routier implique la collecte des données décrivant les caractéristiques des cibles et leur déplacement sur les réseaux routiers. Le nombre des cibles, la vitesse des cibles, le trajet, les débits, la longueur des véhicules, la classe (camion, moyenne véhicule, petite véhicule, moto, piétons) sont autant d'exemples de données utiles. Ces données peuvent être utilisées à l'une des trois cas suivants :

- Application de la loi : Excès de vitesse des véhicules, conduite dangereuse, utilisation illégale des pistes cyclables.
- Congestion et détection des incidents : Les files d'attente, les accidents et les vélos sont potentiellement dangereux pour les véhicules qui s'approchent. Si de tels incidents peuvent être détectés, des panneaux à messages variables et des limites de vitesse peuvent être installés en amont afin d'avertir les conducteurs qui s'approchent.
- Augmenter la capacité des routes : Augmenter la capacité des routes existantes est une alternative intéressante à la construction de nouvelles routes. Si l'on dispose d'informations suffisantes sur l'état d'un réseau routier, il est possible d'acheminer automatiquement le trafic sur les routes les moins encombrées à une vitesse contrôlée afin d'optimiser la capacité globale du réseau.

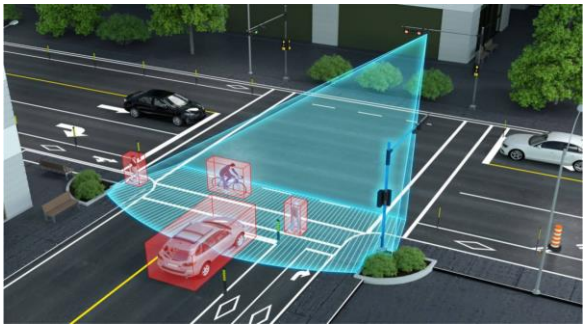


*Figure 1 . 1: Exemple de reconnaissance de cibles radar pour les applications de surveillance du trafic*

### 1.2.2 Techniques de reconnaissance des cibles

La classification des objets en mouvement basée sur les données des capteurs n'est pas un nouveau domaine. En fait, il existe plusieurs techniques de reconnaissance de cibles dans les zones urbaines utilisant différentes technologies. Cependant, le processus de classification se fait généralement à un niveau élevé, en combinant les données d'entrée recueillies par plusieurs capteurs tels que le LIDAR, les caméras stéréo et les radars.

L'utilisation de plusieurs capteurs et la fusion des données des capteurs peuvent fournir une classification très précise, mais nécessitent également une forte puissance de traitement. En outre, l'utilisation de plusieurs capteurs entraînera un coût plus élevé pour la plate-forme de capteurs, et les pièces mobiles que l'on trouve dans les systèmes LIDAR peuvent entraîner une réduction de la robustesse matérielle du système.

Parmi les méthodes basées sur les capteurs, la reconnaissance de cibles mobiles par radar a attiré beaucoup d'attention pour les raisons suivantes. Premièrement, le radar est robuste à la lumière et aux conditions météorologiques, il peut donc être utilisé dans des environnements difficiles. Deuxièmement, le radar pourrait protéger la vie privée visuelle. Au lieu de capturer la forme visuelle de la cible, les signaux renvoyés modulés par la cible transportent d'abondantes informations sur la distance et la vitesse des cibles, qui varient dans le temps. Enfin, le radar est capable de détecter les cibles à travers les murs, ce qui rend la reconnaissance des cibles mobiles par radar applicable à un plus grand nombre de scénarios. Par conséquent, le radar a été adopté de plus en plus souvent pour la reconnaissance de cibles mobiles pour la surveillance du trafic, récemment.

## 1.3 Objectifs du projet et méthodologie de conception

### 1.3.1 Objectifs du projet

Ce projet de stage vise à développer et à évaluer une solution de classification par algorithmes DL dans le but d'identifier les objets mobiles à partir du radar FMCW fonctionnant à24Ghz.

Les objets détectés doivent être classés selon ces cinq classes :

- Piétons
- Moto
- Petite véhicule
- Moyenne véhicule
- Camion

### 1.3.2 Planification du projet

La liste suivante résume les principales activités du projet :

1. Analyse de la base de données en utilisant les outils appropriés
2. Etude comparative des algorithmes DL par rapport aux caractéristiques des données et les exigences de l'application
3. Justifier le choix de solution algorithmique
4. Dimensionner la solution algorithmique
5. Programmation de la solution IA en utilisant les librairies Phyton open source
6. Evaluation des performances et optimisation des paramètres des algorithmes

## 1.4 Environnement de travail du projet de stage

### 1.4.1 Présentation de l'entreprise d'accueil

**Wevioo holding**, créée en 1998 et certifiée ISO 9001 V2001, spécialisée dans les services informatiques, le conseil et l'innovation technologique, fournit à ses clients des solutions d'innovation numérique parfaitement adaptées à leurs enjeux d'agilité, de performance et de développement. Ainsi, avec une culture de l'innovation, Wevioo investit dans la R&D pour apporter à ses clients et partenaires l'expertise des technologies les plus récentes et des solutions les plus innovantes.




*Figure 1 . 2: Logo WEVIOO*

Le groupe Wevioo possède cinq filiales implantées à Tunis, Paris, Alger, Dubaï et Abidjan. Le groupe rassemble plus de 400 collaborateurs, dont 60 % des hommes et 40 % des femmes.

Wevioo réalise des projets pour des clients du monde entier : 60% en Europe, 25% en Afrique et 15% au Moyen-Orient.




*Figure 1 . 3: Présence de WEVIOO dans le monde*

## 1.4.2 Outils matériels et logiciels

### Google Colaboratory

Tout entraînement de réseau de neurones nécessite une forte puissance de calcul. En particulier, notre système est basé sur un réseau de neurones profond et un réseau convolutionnel profond, ce qui implique un grand nombre de poids à entraîner. L'entraînement du modèle à l'aide d'un PC portable classique aurait pris beaucoup trop de temps, c'est pourquoi nous avons décidé d'utiliser la plateforme Google Colaboratory. Il s'agit d'un environnement de notebook Jupyter qui s'exécute directement dans le Cloud et a l'avantage de mettre des GPU à disposition des utilisateurs. La plupart des librairies y sont préinstallées, et il est possible d'en ajouter simplement. La plateforme est liée à Google Drive, il nous a donc suffit d'uploader nos bases de données pour pouvoir les utiliser, et d'exporter nos réseaux entraînés en fin de session. Cette technologie nous a permis de gagner énormément de temps.



*Figure 1 . 4: Google Colab*

**Matlab**

Nous avons utilisé Matlab pour générer les données de test et d'apprentissage basées sur la simulation à partir de modèles de systèmes physiques exprimés dans MATLAB et Simulink®. D'autre part nous avons utilisé Matlab pour la labellisation et la conversion de données Matlab du format .mat en format .csv pour une utilisation sous tensorflow.




*Figure 1 . 5: Matlab & Simulink*

**Python et ses librairies**

Nous avons utilisé Python qui est un langage de programmation interprété, de haut niveau et d'usage général. Python, créé par Guido van Rossum et publié pour la première fois en 1991, a une philosophie de conception qui met l'accent sur la lisibilité du code, notamment en utilisant des espaces blancs importants. Il fournit des constructions qui permettent une programmation claire à petite et grande échelle.




*Figure 1 . 6: Python*

Nous avons choisi Python pour sa grande capacité à manipuler des données et pour la disponibilité de divers outils de développement qui peuvent être utilisés pour travailler avec d'autres systèmes. Nous avons utilisé pour le traitement de la classification les bibliothèques Python suivantes :

1. Pandas pour la manipulation et l'analyse des données
2. Numpy pour les opérations mathématiques et logiques sur les tableaux de données
3. Keras pour la construction de modèles DL
4. Matplotlib et plotly pour la visualisation des données

## 1.5 Conclusion

Dans ce chapitre d'introduction, nous avons donné un aperçu de la reconnaissance des cibles pour l'application de surveillance du trafic à l'aide du radar FMCW à 24 GHz. Nous présentons également la méthodologie et les exigences du projet. Ensuite, nous avons présenté le cadre que nous avons utilisé dans ce projet.

# Chapitre 2 : Aspects de la conception de la reconnaissance des cibles radar

## 2.1 Introduction

Dans ce chapitre, qui est réservé aux considérations de conception pour la reconnaissance des cibles radar, nous commençons par un bref aperçu sur le radar FMCW 24 GHZ et finissons par définir quelques concepts théoriques et comprendre des techniques basées sur l'intelligence artificielle (IA) pour la classification des cibles radar.

## 2.2 Radar FMCW 24GHZ

Un radar à onde continue modulée en fréquence FMCW 24 GHz émet une impulsion électromagnétique modulée en conséquence en changeant sa fréquence pendant le fonctionnement. Cette modulation peut être de type rampe linéaire, modulation par déplacement de fréquence ou Doppler. Comme le système radar sera principalement examiné avec une modulation de rampe linéaire.

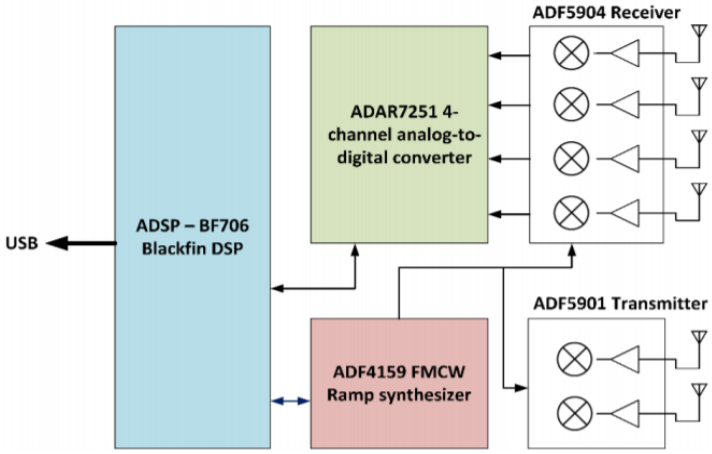



*Figure 2 . 1: Composants de l'émetteur-récepteur radar DemoRad*

Pour faciliter l'appréhension de la détection de cibles basée sur les FMCW, la constellation du signal émis et reçu constitue également un cas particulier, car le système radar et l'objet détecté se déplacent à la même vitesse et dans la même direction. Si l'objet détecté se déplace à une vitesse et dans une direction différente, le signal reçu sera décalé en fréquence, ce que l'on appelle le décalage Doppler. Le système radar émet un signal modulé en fréquence avec un temps de rampe spécifique T et une certaine largeur de bande B. Il reçoit également un signal d'écho avec un décalage de fréquence Δf1 après une durée spécifique Δtp.

La figure ci-dessous illustre l'architecture des étapes de traitement de la détection.

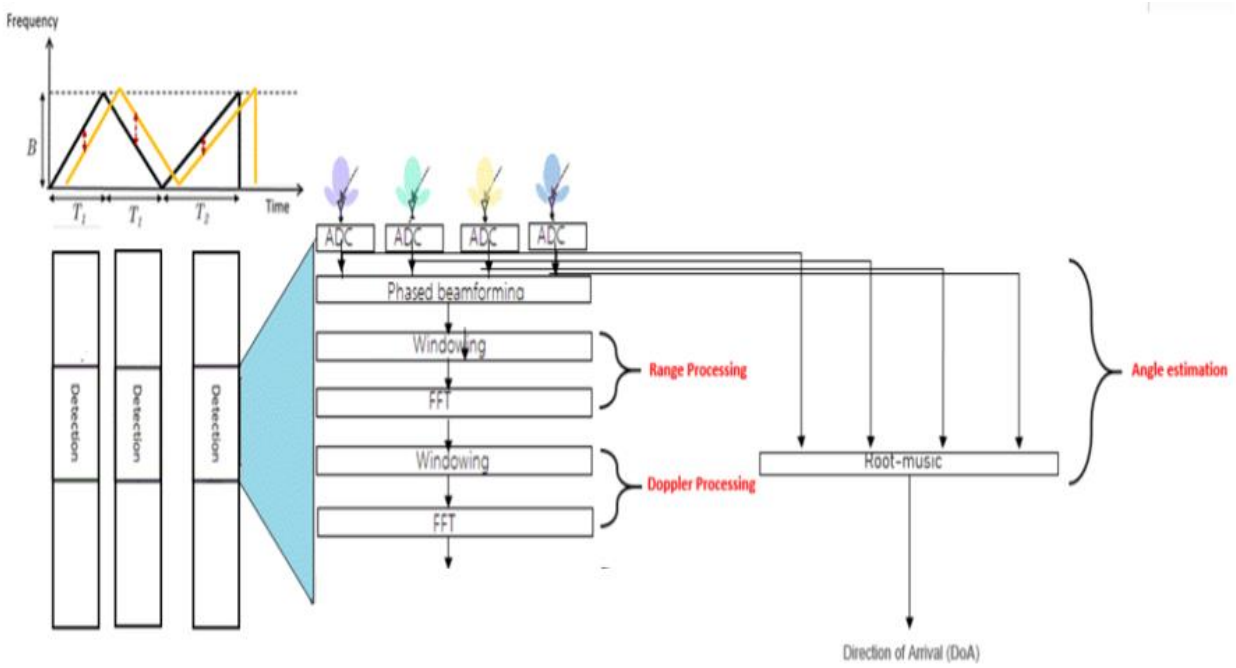



*Figure 2 . 2: Architecture des étapes de traitement de la détection des cibles*

- Traitement de formation de faisceau (Beam forming processing) : l'algorithme de formation de faisceau par déphasage numérique (DBF) est appliqué pour combiner le signal de puissance reçu provenant de 4 antennes dans le domaine temporel.
- Traitement de la portée (Range processing): Pour le signal résultant de l'étape précédente, un fenêtrage 1-D et une transformée de Fourier rapide (FFT) 1-D sont appliqués. Le traitement de la portée est entrelacé avec le temps d'échantillonnage actif de la trame.
- Traitement Doppler (Doppler processing) : Après l'étape de traitement de la portée, un fenêtrage 2D et une FFT 2D sont appliqués. Pour chaque point détecté dans la portée et l'espace Doppler, reconstruire la sortie de la FFT 2D (carte RD) avec compensation Doppler.

## 2.3 Réseau de neurones

Définition : Le réseau de neurones artificiels (ANN) est un système dont la conception est à l'origine schématiquement inspirée du fonctionnement des neurones biologiques, et qui par la suite s'est rapproché des méthodes statistiques. Ils représentent une branche de l'intelligence artificielle (IA) et résolvent des problèmes impossibles ou difficiles à résoudre selon les normes humaines ou statistiques. Les ANN ont des capacités d'auto-apprentissage qui leur permettent de produire de meilleurs résultats à mesure que de nouvelles données sont disponibles.

**Comment ça marche ?**

Les ANN utilisent des centaines ou des milliers de neurones artificiels, qui sont des nœuds contenant un nombre compris entre 0 et 1. Ce nombre représente la probabilité d'être un neurone activé (objet ou non dans la détection d'objets). Les ANN commencent par un neurone d'entrée qui définit réellement l'entrée à résoudre, représenté en neurones (ex : niveau de gris des pixels). Les couches de neurones suivantes sont la sortie de fonctions qui prennent comme entrée la couche de neurones précédente, et sont appelées les couches cachées. La dernière couche est la couche de sortie qui représente le résultat du réseau de neurones artificiels qui est le neurone activé dans la couche de sortie. Les fonctions entre les couches sont linéaires fonctions : Wi*Xi+B : dans laquelle Wi est le poids donné pour chaque nœud d'entrée Xi, B est un seuil pour être activé ou non. La couche suivante de neurones est obtenue par cette fonction :

$$Y\,j = f(X(W\,i * Xi + Bi)) \tag{1.1}$$

dans laquelle Yj est le neurone de sortie et f est une fonction permettant de retourner le résultat entre 0 et 1 (ex : sigmoïde). L'intelligence des ANN consiste donc à déterminer les poids à utiliser pour obtenir le bon résultat pour les différentes entrées. Et pour cela, nous devons former ce réseau avec plusieurs données avant de se faire tester.

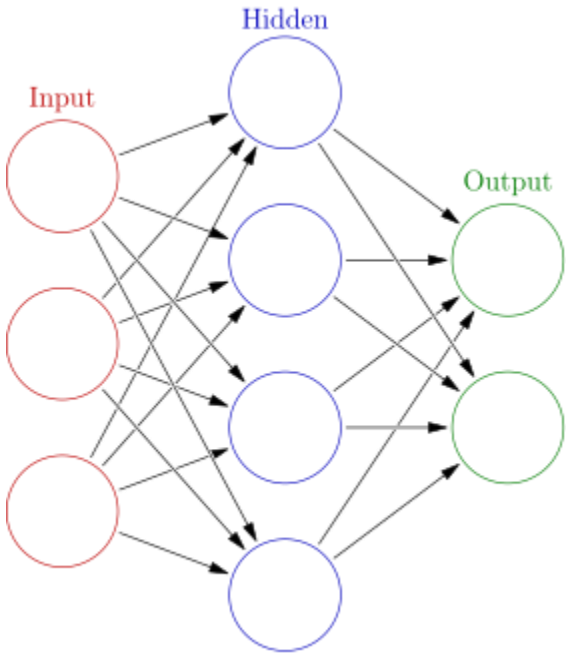



*Figure 2 . 3: Vue simplifiée d'un réseau de neurones artificiels*

## 2.4 Réseau neuronal convolutif

Un CNN fonctionne bien dans les données pour identifier des motifs simples qui seront ensuite utilisés pour créer des motifs de plus en plus compliqués dans les couches supérieures.

L'objectif principal de l'utilisation d'un tel algorithme est qu'il ne nécessite pas d'étape d'ingénierie des caractéristiques car il apprend les caractéristiques de haut niveau à partir des données de manière incrémentielle.

Un modèle simple de réseau neuronal convolutif (CNN) en 2D est conçu en recourant à keras avec des bibliothèques de tensorflow en utilisant le spectre Doppler en entrée. Nous avons choisi la convolution 2D parce que le déplacement du filtre à travers le spectre se fait en deux dimensions.

Techniquement, chaque matrice d'entrée passera à travers une série de couches de convolution avec filtres (Kernals), Pooling, couches entièrement connectées (FC) et appliquera la fonction Softmax pour classer un objet avec des valeurs probabilistes entre 0 et 1.

La figure ci-dessous est un flux complet de CNN pour traiter une matrice d'entrée et classifier les objets en fonction des valeurs.

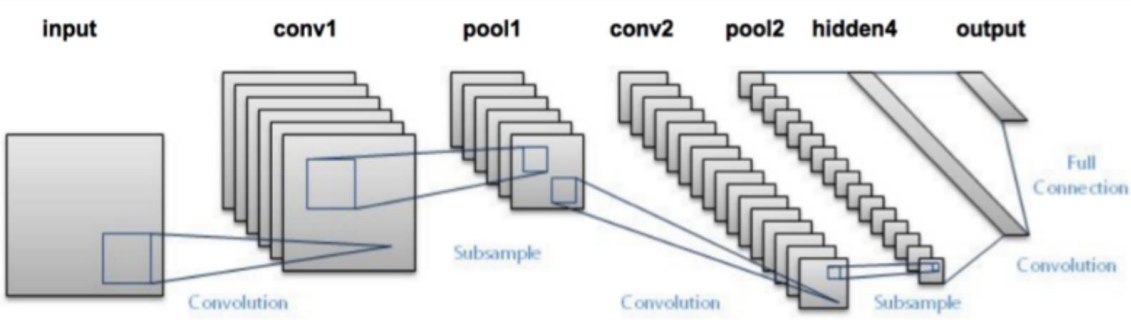



*Figure 2 . 4: Exemple d'un réseau neuronal convolutif en 2D*

**Couche de convolution**

La convolution est la première couche qui permet d'extraire des caractéristiques d'une image d'entrée. La convolution préserve la relation entre les pixels en apprenant les caractéristiques de l'image à l'aide de petits carrés de données d'entrée. C'est une opération mathématique qui nécessite deux entrées telles qu'une matrice d'image et un filtre ou un kernal.

- An image matrix (volume) of dimension **(h x w x d)**
- A filter **($f_h$ x $f_w$ x d)**
- Outputs a volume dimension **(h - $f_h$ + 1) x (w - $f_w$ + 1) x 1**




*Figure 2 . 5: Matrice d'entrée multipliée par le kernel ou la matrice de filtrage*

Le réseau permettra aux filtres de reconnaître une partie de l'objet en détectant les bords et les couleurs sur la première couche, puis l'objet entier sur les couches supérieures. Les dimensions (largeur et hauteur) des filtres sont plus petites que l'image d'entrée.

**Regroupement des couches (Pooling Layers)**

Après une série de couches convolutionnelles dans l'architecture de CNN, nous insérons une couche de regroupement pour réduire la taille spatiale de la représentation, diminuer les paramètres et donc aussi contrôler le surajustement (overfitting). La couche de pooling fonctionne indépendamment sur chaque tranche de profondeur en utilisant l'opération max.

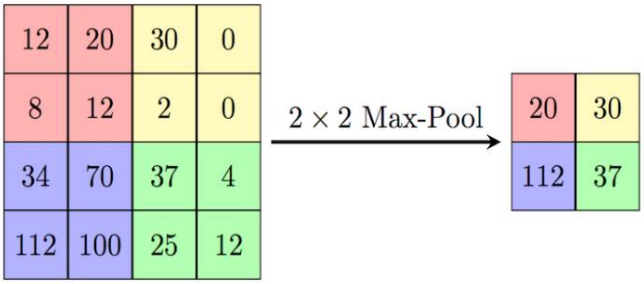



*Figure 2 . 6: Couche de Max polling 2*2*

**Couches entièrement connectées**

La couche entièrement connectée est essentiellement constituée des réseaux neuronaux réguliers que les neurones sont activés par accréditation sur la couche précédente.

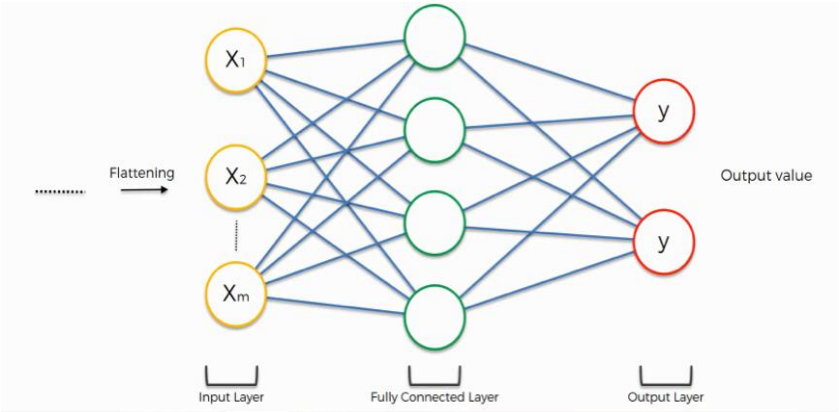



*Figure 2 . 7: Couche entièrement connectée dans un CNN*

## 2.5 Conclusion

Dans ce chapitre, nous avions défini les concepts nécessaires et fondamentaux pour construire notre modèle. Et après avoir compris les définitions théoriques, il faut maintenant passer au travail concret.

# Chapitre 3 : Exploration des données

## 3.1 Introduction

La phase de traitement et de préparation des données est une étape importante dans le développement d'un algorithme d'apprentissage profond. En effet, elle permet de mieux comprendre l'application à réaliser afin de développer correctement le modèle. En conséquence, une étude approfondie a été établie pour identifier les meilleurs ensembles de données sur lesquels nous allons tester notre modèle au fur et à mesure que nous les détaillons dans ce chapitre.

## 3.2 Présentation de l'ensemble des données

Notre base de données est formé par 2100 trames avec un format standard défini comme une matrice 256×128 qui sont pris après le bloc 2D-FFT. Cette base de données est composée de 2000 échantillons de données de formation et de 100 échantillons de données d'essai et chaque échantillon de ces données correspond à la densité spectrale de puissance d'une cible parmi ces 5 cibles (piétons, moto, petite véhicule, moyenne véhicule, camion).

```
[ ]  classes = ['Pedestrian','Motorcycle','Small_Vehicle','Medium_Vehicle', 'Truck'] # our different target
```

*Figure 3 . 1: Classes des objets détectés et classifiés*

## 3.3 Nettoyage des données en double

La "duplication" signifie simplement que nous avons des données répétées dans notre base de données. Cela peut être dû à des éléments tels que des erreurs de saisie ou des méthodes de génération de données dans Matlab. Quelle qu'en soit la raison, la déduplication peut nous amener à tirer des conclusions erronées en nous faisant croire que certaines observations sont plus fréquentes qu'elles ne le sont en réalité. L'apparition de doublons ne signifie pas nécessairement que l'ensemble des données est complètement erroné - seulement que l'ensemble de données peut nécessiter un examen plus approfondi et un travail de nettoyage supplémentaire, comme c'est le cas pour la plupart des ensembles de données.

**Data cleaning :**

- Identify Rows that Contain Duplicate Data
- Delete Rows that Contain Duplicate Data

```
[ ]  # calculate duplicates
     dups = X.duplicated()
     # report if there are any duplicates
     print(dups.any())
     # delete duplicate rows
     print("X_ train shape before deleting duplicate values : "+str(X.shape))
     X.drop_duplicates(inplace=True)
     print("X_ train shape after deleting duplicate values : "+str(X.shape))
```

```
True
X_ train shape before deleting duplicate values : (1999, 32768)
X_ train shape after deleting duplicate values : (1759, 32768)
```

*Figure 3 . 2: Nettoyage des données d'apprentissage*

## 3.4 Augmentation de la base des données

L'augmentation des données d'apprentissage est une technique qui peut être utilisée pour augmenter artificiellement la taille d'un ensemble de données de formation en créant des versions modifiées des images dans l'ensemble de données.

La formation de modèles de réseaux neuronaux d'apprentissage profond sur un plus grand nombre de données peut donner lieu à des modèles plus performants, et les techniques d'augmentation peuvent créer des variations des images qui peuvent améliorer la capacité des modèles d'ajustement à généraliser ce qu'ils ont appris à de nouvelles images. En effet, dans notre cas nous avons fait un simple renversement horizontal et vertical des matrices d'apprentissage comme ce qui est montré dans la figure ci-dessous.

```
[12] #training dataset augmentation
     X=np.reshape(X, (X.shape[0],256,128))
     X=np.concatenate((X, X[:,:,::-1]), axis=0) # flip the train array horizontally
     X=np.concatenate((X, X[:,::-1,:]), axis=0) # flip the train array vertically
     X=np.reshape(X, (X.shape[0],32768))
     Y=np.concatenate((Y,Y,Y,Y), axis=0)
```

```
[13] print("X_ train shape after data aungmentation process  : "+str(X.shape))
     print("Y_ train shape after data aungmentation process  : "+str(Y.shape))
```

```
X_ train shape after data aungmentation process  : (7036, 32768)
Y_ train shape after data aungmentation process  : (7036, 1)
```

*Figure 3 . 3: Augmentation de la base des données*

## 3.5 Visualisation des données

Pour la visualisation des données nous avons utilisé Pie chart ce qui est un graphique statistique circulaire qui est divisé en tranches pour illustrer la proportion numérique ou le pourcentage représenté par chaque catégorie dans l'ensemble de données d'apprentissage.

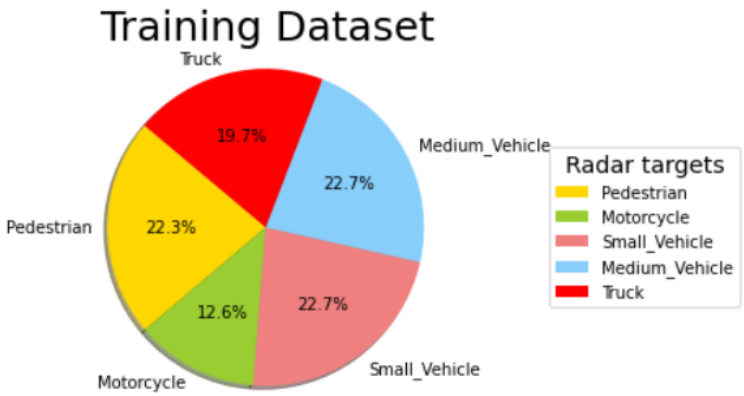



*Figure 3 . 4: Représentation des proportions de l'ensemble des données*

Notre ensemble de données est composé par des différentes représentations de la densité spectrale de puissance (DSP) du signal réfléchi par l'objet détecté et qui ça sera par la suite classifié. En fait ces données-là sont la sortie du block 2D_FFT ce qui est mentionné dans le chapitre précédant.

```
y = 0   => It is a representation of a Pedestrian from the training dataset.
```

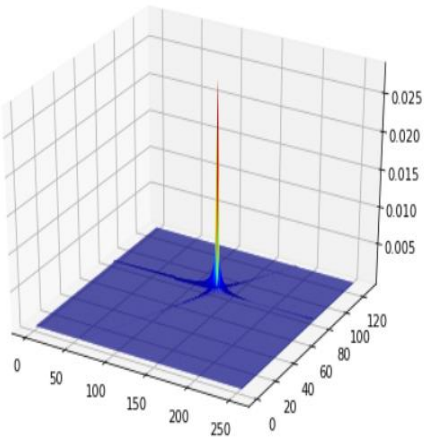



*Figure 3 . 5: Exemple d'une DSP d'un piéton*

```
y = 3   => It is a representation of a Medium_Vehicle from the training dataset.
```

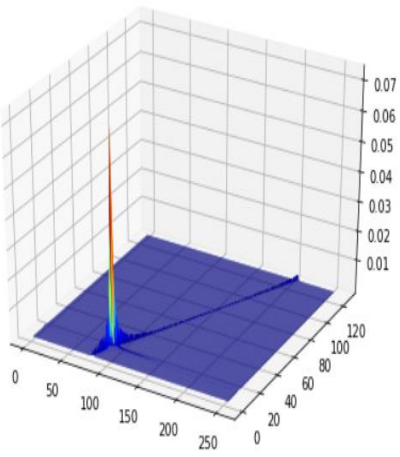



*Figure 3 . 6: Exemple d'une DSP d'une moyenne voiture*

## 3.6 Normalisation des données

La normalisation Minmax est une stratégie de normalisation qui transforme linéairement x en

$$x' = (x-min)/(max-min)$$

où min et max sont les valeurs minimales et maximales en X, où X est l'ensemble des données d'apprentissage. Lorsque x=max, alors x'=1. Cela signifie que la valeur minimale en X est mise en correspondance avec 0 et la valeur maximale en X est mise en correspondance avec 1. Si nous effectuons cette technique de normalisation, l'ANN et le CNN fonctionnent plus rapidement.

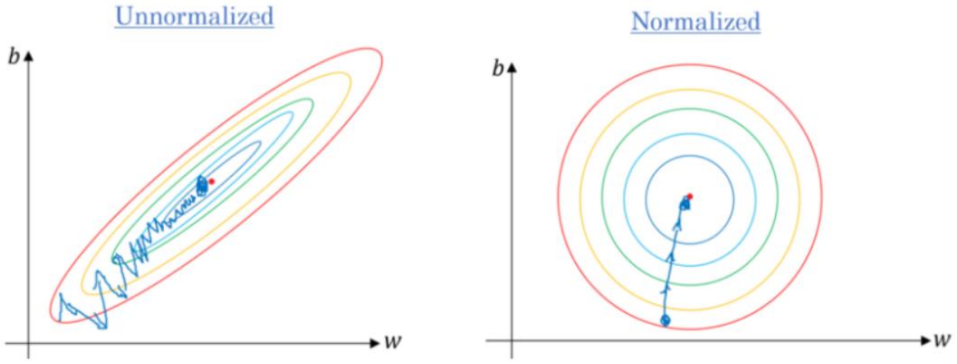



*Figure 3 . 7: Effet de la normalisation des données*

## 3.7 encodage des labels

L'encodage est le processus de conversion des données dans un format requis afin de les alimenter au modèle : nous avons plusieurs méthodes et algorithmes d'encodage ; dans notre approche, nous avons choisi la méthode to_categorical ce qui permet de convertir un vecteur de classe (nombres entiers) en matrice de classe binaire.

**Label Encoding:**

- Encode labels to one hot vectors
  - 0 => [1,0,0,0,0] : pedestrian
  - 1 => [0,1,0,0,0] : motorcycle
  - 2 => [0,0,1,0,0] : small vehicle
  - 3 => [0,0,0,1,0] : medium vehicle
  - 4 => [0,0,0,0,1] : truck

*Figure 3 . 8: Encodage des labels*

## 3.8 Conclusion

Dans ce chapitre, nous avons décrit les données et nous avons présenté les techniques utilisées pour les préparer à alimenter le modèle qui sera décrit dans le chapitre suivant.

# Chapitre 4 : Description des modèles

## 4.1 Introduction

Après avoir présenté dans le chapitre précédent le prétraitement des données mesurées, le contenu de ce chapitre concerne la description de la conception des modèles pour les techniques d'apprentissage approfondi. La première section de ce chapitre présentera les mesures de développement et d'évaluation des modèles basés sur le ANN. La deuxième section illustrera le développement du modèle DL pour l'approche CNN et finira par l'évaluation de ce modèle.

## 4.2 Développement du modèle basé sur l'ANN

Keras est un outil simple pour construire un réseau de neurones. Il s'agit d'un cadre de haut niveau basé sur les backends tensorflow, theano ou cntk.

Dans notre ensemble de données, l'entrée est de 32768 valeurs ce qui était arrivé de la vectorisation de la matrice (256*128) et la sortie de 5 valeurs (nombre des classes: piétons, moto, petite véhicule, moyenne véhicule, camion). Ainsi, les couches d'entrée et de sortie ont respectivement 32768 et 5 dimensions.

```
[ ]  # Build the model
     model1 = Sequential()
     model1.add(Input(shape=(32768,)))
     model1.add(Dense(1024,activation='relu', activity_regularizer=l2(0.001)))
     model1.add(Dropout(0.5))
     model1.add(Dense(2048,activation='relu' , activity_regularizer=l2(0.001)))
     model1.add(Dropout(0.5))
     model1.add(Dense(1024,activation='relu' , activity_regularizer=l2(0.001)))
     model1.add(Dropout(0.5))
     model1.add(Dense(1024,activation='relu', activity_regularizer=l2(0.001)))
     model1.add(Dropout(0.5))
     model1.add(Flatten())
     model1.add(Dense(5, activation='softmax'))
```

*Figure 4 . 1: Modèle ANN*

Dans notre réseau de neurones, nous utilisons quatre couches cachées de 1024, 2048, 1024 et 1024 dimensions. Je vais maintenant expliquer le code ligne par ligne.

Sequential spécifie aux keras que nous créons le modèle séquentiellement et la sortie de chaque couche que nous ajoutons est l'entrée dans la couche suivante que nous spécifions.

model.add est utilisé pour ajouter une couche à notre réseau de neurones. Nous devons spécifier comme argument le type de couche que nous voulons. La Dense est utilisée pour spécifier la

couche entièrement connectée. Les arguments de Dense sont la dimension de sortie qui est 1024 dans le premier cas, la dimension d'entrée qui est 32768 pour la dimension d'entrée et la fonction d'activation à utiliser qui est relu dans ce cas et qui est très efficace sur le plan du calcul. Ajoutons à cela la régularisation L2 (activity regularizer) qui fait l'ajout d'un terme à l'équation de la fonction d'erreur, destiné à diminuer les poids et les biais, à lisser les sorties et à rendre le réseau moins susceptible de se surcharger(overfit). On additionne après chaque couche entièrement connectée (Dense) une couche Dropout qui "tue" aléatoirement un certain pourcentage de neurones (=0.5 %) à chaque itération d'entraînement. Cela permet de s'assurer que certaines informations apprises sont retirées de façon aléatoire, ce qui réduit le risque de sur apprentissage (overfitting). La deuxième couche Dense est similaire, nous n'avons pas besoin de spécifier la dimension d'entrée car nous avons défini le modèle comme étant séquentiel, donc Keras considérera automatiquement que la dimension d'entrée est la même que la sortie de la dernière couche, c'est-à-dire 1024. Dans la dernière couche (couche de sortie), la dimension de sortie est de 5 (nombre de classes). Maintenant, comme nous l'avons vu précédemment, la couche de sortie prend différentes fonctions d'activation et pour le cas de la classification multiclasse, il s'agit de softmax. Softmax prend la sortie de la couche dense et la convertit en probabilités significatives pour chacun des classes, dont la somme est égale à 1. Elle fait ensuite une prédiction basée sur le type de cible qui a la plus grande probabilité.

Nous devons maintenant spécifier la fonction de perte et l'optimiseur. Cela se fait en utilisant la fonction de compilation en keras.

```
[29] model1.compile(loss=categorical_crossentropy,
                    optimizer="Adam",
                    metrics = ['accuracy'])
```

*Figure 4 . 2: Compilation de modèle ANN*

Ici, la perte est une perte d'entropie croisée, Categorical_crossentropy précise que nous avons plusieurs classes. L'optimiseur que nous avons utilisé est Adam ce qui est utile pour sélectionner le taux d'apprentissage (the learning rate) et pour mettre à jour les pondérations du réseau (the network weights). Metrics est utilisé pour spécifier la façon dont nous voulons juger la performance de notre réseau de neurones. Ici, nous l'avons spécifié avec précision(accuracy). Cette fonction métrique est similaire à la fonction de perte, sauf que les résultats de l'évaluation métrique ne sont pas utilisés lors de la formation du modèle (uniquement pour l'évaluation).

L'étape de formation ou d'entrainement du modèle est simple dans keras. model.fit est utilisé pour le former.

```
[25] history = model1.fit(X_train1, Y_train1, batch_size=16,epochs=40,
                          validation_data=(X_test1, Y_test1))
```

*Figure 4 . 3: Entrainment de modèle ANN*

Ici, nous devons spécifier les données d'entrée-> X_train1, labels-> y_train1, le nombre d'époques (itérations) et la taille du lot. Il retourne l'historique de la formation du modèle. L'historique consiste en la précision du modèle et les pertes après chaque époque. Nous la visualiserons plus tard.

Nous avons utilisé les données des tests comme données de validation et nous vérifions les précisions après chaque époque. Cela nous donnera un aperçu du sur-ajustement au moment de la formation uniquement et nous pourrons prendre des mesures avant l'achèvement de toutes les époques.

Habituellement, l'ensemble de données est très grand et nous ne pouvons pas ajuster des données complètes en une fois, donc nous utilisons la taille des lots. Cela divise nos données en lots dont la taille est égale à batch_size. Maintenant, seul ce nombre d'échantillons sera chargé en mémoire et traité. Une fois que nous avons terminé un lot, il est effacé de la mémoire et le lot suivant sera traité.

À l'aide de la méthode "Cross_Validation" j'ai obtenu les meilleurs résultats avec une taille de "Batch" de 16 parmi cette liste de batch size:[8,16,32,64,128].

J'ai fixé le nombre d'époques à 40 ce qui est le plus élevé nombre après lequel l'erreur de validation commence à augmenter, et cela pourrait être une indication d'un sur-ajustement.

```
Epoch 1/40
112/112 [==============================] - 20s 183ms/step - loss: 1.3637 - accuracy: 0.3522 - val_loss: 0.9041 - val_accuracy: 0.6159
Epoch 2/40
112/112 [==============================] - 20s 181ms/step - loss: 0.8182 - accuracy: 0.6669 - val_loss: 0.8946 - val_accuracy: 0.5270

                                                ...

Epoch 38/40
112/112 [==============================] - 22s 196ms/step - loss: 0.0613 - accuracy: 0.9871 - val_loss: 0.2593 - val_accuracy: 0.9556
Epoch 39/40
112/112 [==============================] - 22s 195ms/step - loss: 0.0745 - accuracy: 0.9826 - val_loss: 0.1958 - val_accuracy: 0.9556
Epoch 40/40
112/112 [==============================] - 22s 194ms/step - loss: 0.0434 - accuracy: 0.9905 - val_loss: 0.1845 - val_accuracy: 0.9651
```

*Figure 4 . 4: Phase d'entrainement*

Il faut environ une 15 minute pour s'entraîner. Et après 40 époques, le réseau de neurones sera entraîné. La précision de la formation est de 99,05% et la précision des tests est de 96.51%, donc notre modèle est entraîné.

Notre modèle fonctionne bien. Nous allons maintenant visualiser les pertes et les précisions de la formation et de la validation.

```
[28] plt.plot(history.history['accuracy'], label='accuracy')
     plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
     plt.xlabel('Epoch')
     plt.ylabel('Accuracy')
     plt.ylim([0, 1])
     plt.title("Accuracy curve", fontsize=25)
     plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7ff852fd2cf8>
```

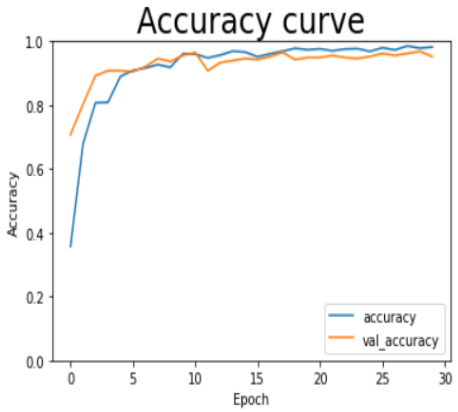



*Figure 4 . 5: Courbes de précision*

```
[29] plt.plot(history.history['loss'], label='loss')
     plt.plot(history.history['val_loss'], label = 'val_loss')
     plt.xlabel('Epoch')
     plt.ylabel('Loss')
     plt.ylim([0, 1.75])
     plt.title("Loss curve", fontsize=25)
     plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7ff859e57d68>
```

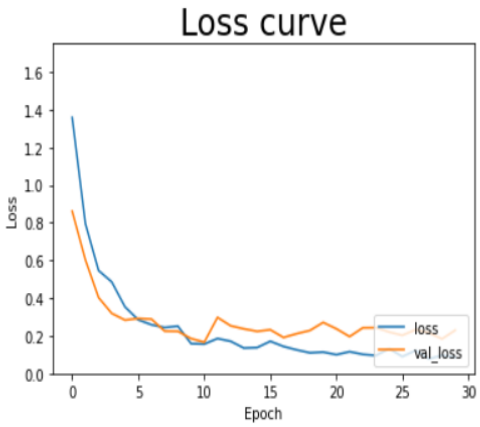



*Figure 4 . 6: Courbes de perte*

La meilleure façon d'évaluer la performance d'un classificateur est de regarder la matrice de confusion(CM). L'idée générale est de compter le nombre de fois que des instances de la classe A sont classées dans la classe B. Chaque ligne de la matrice reflète les cas dans une classe, tandis que chaque colonne représente les cas dans une classe prévue. Ce nom vient du fait qu'il est simple de voir si le régime confond deux classes.

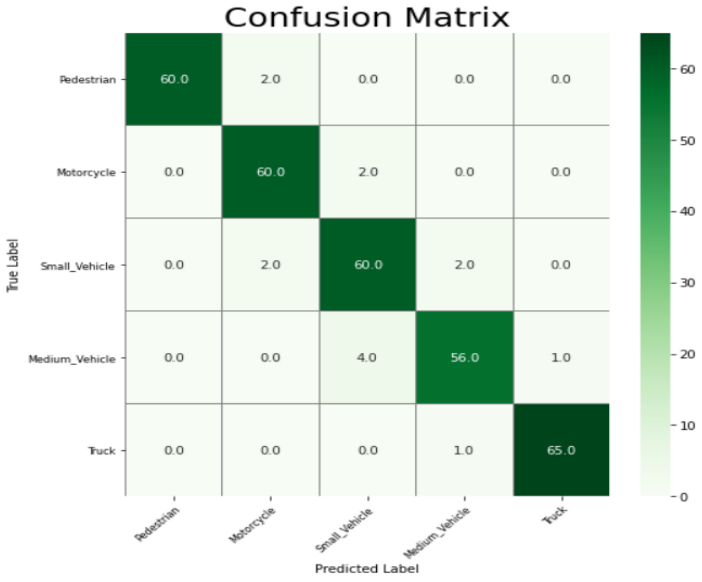



*Figure 4 . 7: Matrice de confusion de modèle ANN*

## 4.3 Développement du modèle basé sur le CNN

L'architecture globale du réseau est composée d'une étape d'entrée, d'une étape d'extraction des caractéristiques avec cinq blocs de couches de convolution et d'une étape de classification des dispositions de sortie. L'étage d'entrée reçoit l'ensemble de données prétraitées. L'étape d'extraction des caractéristiques a suivi une structure de blocs de convolution ternaire en quintuplets comprenant des fonctions de convolution, de regroupement et d'activation. La dernière étape de classification consistait en une couche entièrement connectée et un opérateur softmax. Les sorties finales du réseau provenaient des résultats de l'opération softmax.

- **Étape de la convolution**
  - (i) Couche d'entrée : Dans un CNN, une couche peut être représentée par l'expression générale fonction y = f(x)où l'entrée x et la sortie y sont toutes deux des tenseurs. L'entrée du réseau est une donnée tensorielle de taille Hauteur(H) Largeur(W) Dimension(D). Dans ce modèle la taille d'entrée H W D était égale à $256 * 128 * 1$.
  - (ii) Couche de convolution : La couche de convolution a calculé la convolution x d'entrée en utilisant des filtres f afin de détecter les conjonctions locales des caractéristiques de la couche précédente et de cartographier leur apparence sur une carte des caractéristiques. Cependant, dans cette étude, nous avons considéré un type de couches de convolution avec des noyaux bi-dimensionnels car CNN avec un noyau 2D est capable de capturer la dépendance spatiale sur la portée ainsi que la dépendance locale sur la vitesse à une image.
  - (iii) Couche de mise en commun : Les couches de regroupement sont utilisées pour identifier les caractéristiques les plus importantes et comprimer les tailles des couches par sous-échantillonnage. Dans ce projet, une méthode de mise en commun maximale est utilisée après une étape de convolution afin de réduire les besoins de calcul progressivement dans le réseau ainsi que de minimiser la probabilité de sur-ajustement.
  - (iv) Fonction d'activation : Une fonction d'activation a suivi l'étape de mise en commun maximale dans le bloc de convolution. Une unité linéaire rectifiée (ReLU) a été sélectionnée pour cette fonction afin d'atténuer le problème du gradient de disparition et de réduire le temps d'apprentissage. Toute valeur de sortie négative restant après la couche de mise en commun maximale a été mise à zéro.
  - (v) Blocs de convolution répétés : cinq blocs de convolution ont été utilisés, combinant les étapes de convolution, de mise en commun maximale et de ReLU.
- **Étape de classification:**
  - (i) Couche "Flatten" : une opération d'aplatissement sur un tenseur permet de remodeler le tenseur pour qu'il ait une forme égale au nombre d'éléments contenus dans le tenseur.

(ii) Couche entièrement connectée: Les résultats ReLU de chaque couche, comme pour toutes les sorties de couche, ont été connectés aux nœuds pondérés de la couche suivante.

(iii) Couche Softmax : La fonction softmax a été appliquée à tous les canaux de caractéristiques pour consolider les résultats du réseau. Cette méthode de normalisation a été appliquée pour faire en sorte que la somme des distributions de probabilité de sortie soit égale à 1.

```
[ ] # Build the model
    model = Sequential()
    model.add(Conv2D(filters = 8, kernel_size = (11,11),padding = 'Same', activation ='relu', input_shape = (256,128,1)))
    model.add(MaxPool2D(pool_size=(2,2)))

    model.add(Conv2D(filters = 16, kernel_size = (9,9),padding = 'Same', activation ='relu'))
    model.add(MaxPool2D(pool_size=(2,2)))

    model.add(Conv2D(filters = 32, kernel_size = (7,7),padding = 'Same', activation ='relu'))
    model.add(MaxPool2D(pool_size=(2,2)))

    model.add(Conv2D(filters = 64, kernel_size = (5,5),padding = 'Same', activation ='relu'))
    model.add(MaxPool2D(pool_size=(2,2)))

    model.add(Conv2D(filters = 128, kernel_size = (3,3),padding = 'Same',  activation ='relu'))
    model.add(MaxPool2D(pool_size=(2,2)))

    # fully connected
    model.add(Flatten())
    model.add(Dense(128, activation = "relu"))
    model.add(Dense(128, activation = "relu"))
    model.add(Dense(5, activation = "softmax"))
```

*Figure 4 . 8: Modèle CNN*

Dans Conv2d, nous devons alimenter le nombre de canaux en sortie, ce qui est le premier argument. Par convention, le nombre de canaux augmente généralement ou reste le même alors que nous progressons à travers les couches de notre architecture de réseau neuronal convolutionnel [8->16->32->64->128]. Le second argument est la taille du noyau que nous avons les utilisé ce qui sont respectivement [11,9,7,5,3]. Nous pouvons utiliser une taille différente, mais nous la gardons en général assez importante au début et après on fait les décroitre progressivement pour apprendre des modèles plus complexes, ce qui donne au réseau une plus grande puissance d'expression. Le troisième argument est le type de padding utilisé, nous avons essayé d'utiliser le "Padding = same" car nous avons pensé que les bordures de l'image peuvent être importantes et pour aider à allonger mon architecture de réseau car le "padding" conserve les mêmes dimensions même après l'opération de convolution et nous pouvons donc effectuer plus de convolutions sans réduire la taille. Finalement nous connaissons le quatrième argument, qui est la fonction d'activation ReLU.

Dans Maxpool2d, nous ne donnons qu'un seul argument, la taille du pool et dans ce cas nous avons utilisé des filtres de taille 2x2.

Nous devons maintenant spécifier la fonction de perte et l'optimiseur. Cela se fait en utilisant la fonction de compilation en keras.

```
# complile the model and view its architecture
model.compile(loss=keras.losses.categorical_crossentropy,  optimizer="adam", metrics=['accuracy'])
```

*Figure 4 . 9: Compilation de modèle CNN*

Ici, la perte est une perte d'entropie croisée, Categorical_crossentropy précise que nous avons plusieurs classes. L'optimiseur que nous avons utilisé est Adam ce qui est utile pour sélectionner le taux d'apprentissage(the learning rate) et pour mettre à jour les pondérations du réseau(the network weights). Metrics est utilisé pour spécifier la façon dont nous voulons juger la performance de notre réseau de neurones. Ici, nous l'avons spécifié avec précision(accuracy). Cette fonction métrique est similaire à la fonction de perte, sauf que les résultats de l'évaluation métrique ne sont pas utilisés lors de la formation du modèle (uniquement pour l'évaluation).

L'étape de formation ou d'entrainement du modèle est simple dans keras. model.fit est utilisé pour le former.

```
[ ]  history = model.fit(X_train, Y_train, batch_size=128,epochs=30,
                         validation_data=(X_test, Y_test))
```

*Figure 4 . 10: Entrainement de modèle CNN*

Ici, nous devons spécifier les données d'entrée-> X_train, labels-> y_train, le nombre d'époques (itérations) et la taille du lot. Il retourne l'historique de la formation du modèle. L'historique consiste en la précision du modèle et les pertes après chaque époque. Nous la visualiserons plus tard.

Nous avons utilisé les données de test comme données de validation et nous vérifions l'exactitude après chaque époque. Cela nous donnera une vue d'ensemble de la surcharge au moment de la formation seulement et nous pourrons agir avant que toutes les époques soient terminées.

Habituellement, l'ensemble de données est très important et nous ne pouvons pas ajuster des données complètes en une fois, donc nous utilisons la taille des lots. Nous utilisons donc la taille des lots. Nos données sont ainsi divisées en lots dont la taille est égale à batch_size. Maintenant, seul ce nombre d'échantillons sera chargé en mémoire et traité. Une fois que nous avons terminé un lot, il est effacé de la mémoire et le lot suivant sera traité.

En utilisant la méthode de "Cross_Validation", j'ai obtenu les meilleurs résultats avec une taille de lot de 128 à partir de cette liste de taille de lot : [16,32,64,128,256].

J'ai fixé le nombre d'époques à 30, qui est le nombre le plus élevé après lequel l'erreur de validation commence à augmenter, ce qui pourrait être une indication de surajustement.

```
Epoch 1/30
16/16 [==============================] - 155s 10s/step - loss: 1.5670 - accuracy: 0.2216 - val_loss: 1.5094 - val_accuracy: 0.1616
Epoch 2/30
16/16 [==============================] - 154s 10s/step - loss: 1.3429 - accuracy: 0.3947 - val_loss: 1.2924 - val_accuracy: 0.5253
Epoch 3/30
16/16 [==============================] - 156s 10s/step - loss: 1.2337 - accuracy: 0.4332 - val_loss: 1.1132 - val_accuracy: 0.5960

...

Epoch 28/30
16/16 [==============================] - 158s 10s/step - loss: 0.1089 - accuracy: 0.9650 - val_loss: 0.1586 - val_accuracy: 0.9495
Epoch 29/30
16/16 [==============================] - 156s 10s/step - loss: 0.1136 - accuracy: 0.9635 - val_loss: 0.0916 - val_accuracy: 0.9798
Epoch 30/30
16/16 [==============================] - 156s 10s/step - loss: 0.0383 - accuracy: 0.9870 - val_loss: 0.0288 - val_accuracy: 0.9899
```

*Figure 4 . 11: Phase d'entrainement*

Il faut environ une 80 minute pour s'entraîner. Et après 30 époques, le réseau de neurones sera entraîné. La précision de la formation est de 98,7 % et la précision des tests est de 98.9%, donc notre modèle est entraîné.

Notre modèle fonctionne bien. Nous allons maintenant visualiser les pertes et les précisions de la formation et de la validation.

```
[ ] plt.plot(history.history['accuracy'], label='accuracy')
    plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.ylim([0, 1])
    plt.title("Accuracy curve", fontsize=25)
    plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7f73c30430b8>
```

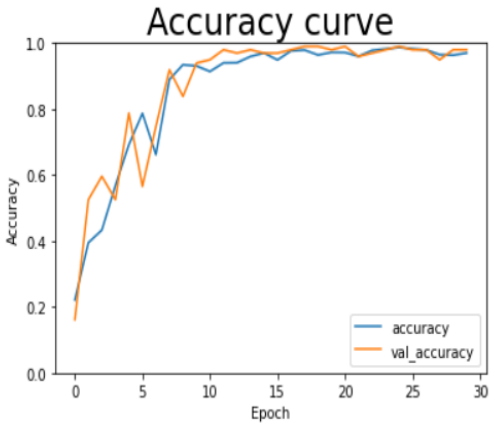



*Figure 4 . 12: Courbes de précision*

```
[ ] plt.plot(history.history['loss'], label='loss')
    plt.plot(history.history['val_loss'], label = 'val_loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.ylim([0, 1.5])
    plt.title("Loss curve", fontsize=25)
    plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7f73c2fbee10>
```

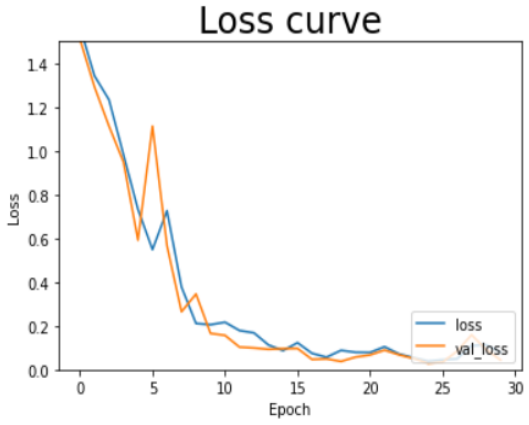



*Figure 4 . 13: Courbes de perte*

Il est clair qu'il n'y a pas de problème de confusion pour identifier toutes les classes comme le montre la figure ci-dessous.

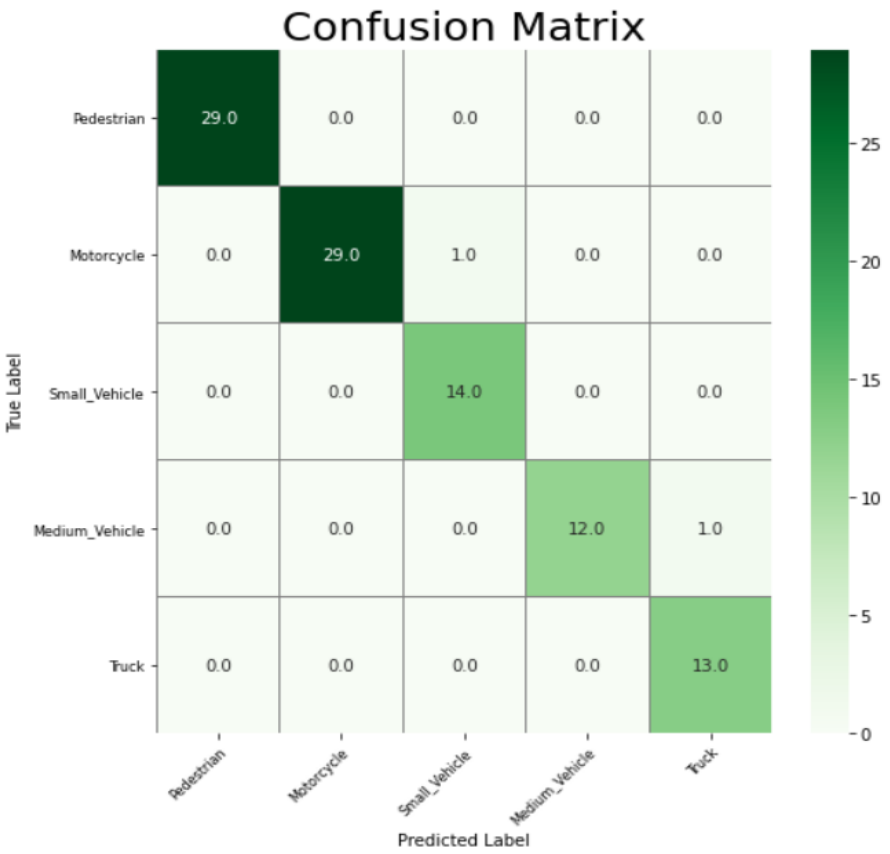



*Figure 4 . 14: Matrice de confusion de modèle CNN*

## 4.3 Conclusion

Dans ce chapitre, nous avons discuté les architectures des modèles ANN et CNN ; nous avons précisé les caractéristiques de chaque couche dans ces modèles et évalué ces performances dans la classification de nos cibles.

# Conclusion et perspectives

Dans cette étude, on a effectué la classification des cibles mobiles pour l'application de surveillance du trafic. Cette étude a impliqué le prétraitement des données, le développement d'algorithmes DL, et finalement l'évaluation de ces modèles-là. Les algorithmes développés ont distingué les cibles mobiles en cinq classes différentes (camions, moyennes véhicules, petites véhicules, motos, piétons).

Ensuite nous avons développé deux modèles de classification avec 2 techniques différentes. La première technique basée sur DL CNN est définie avec des noyaux bi-dimensionnels (2D) utilisés pour extraire la dépendance locale du profil Doppler sur différentes trames capturées par le radar. La deuxième technique basée sur la méthode DL ANN a été développée en utilisant des ensembles de données obtenus après le réarrangement des données initiales.

Les résultats obtenus montrent que l'CNN a permis la meilleure précision de classification dans les données des tests (environ 98%) par rapport aux classificateurs ANN (environ 96%).

D’une autre part, je peux dire honnêtement que le temps passé en stage avec WEVIOO a donné lieu à l'un des meilleurs étés de ma vie. Non seulement j'ai acquis des compétences pratiques, mais j'ai également eu l'occasion de rencontrer de nombreuses personnes fantastiques. L'atmosphère de WEVIOO a toujours été accueillante, ce qui m'a permis de me sentir comme chez moi.

De plus, j'ai eu le sentiment pour la première fois après tout un programme d'études que j'ai fait le bon choix et que je suis sur la bonne voie pour réussir. De plus, au cours de ces deux mois, j'ai l'occasion d'apprendre et de bien comprendre l'intelligence artificielle, ce qui m'aidera dans mes futurs projets d'utilisation de cette technologie innovante qu'est l'IA.

Bien que j'aie pu apprendre beaucoup de la vie normale d'ingénieur, mon stage chez WEVIOO a été un succès. J'ai pu acquérir des compétences pratiques, travailler dans un environnement fantastique et nouer des liens qui dureront toute ma vie. Je ne pourrais pas être plus satisfait.

# Références

[1] “Wevioo consulting.” https://www.wevioo.com/fr.

[2]“Radrafmcw”https://www.radartutorial.eu/02.basics/Radar%20%C3%A0%20onde%20continue%20modul%C3%A9e%20en%20fr%C3%A9quence.fr.html .

[3] Y. H. Xinyu Li and X. Jing, A Survey of Deep Learning-Based Human Activity Recognition in Radar. PhD thesis, School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China.

[4]https://www.researchgate.net/post/How_do_we_choose_the_filters_for_the_convolutional_layer_of_a_Convolution_Neural_Network_CNN

[5] https://stackoverflow.com/questions/55444120/understanding-the-output-shape-of-conv2d-layer-in-keras

[6] https://machinelearningmastery.com/convolutional-layers-for-deep-learning-neural-networks/

[7] https://machinelearningmastery.com/evaluate-performance-deep-learning-models-keras/

[8] https://www.quora.com/How-do-I-increase-accuracy-using-Convolutional-Neural-Networks-CNNs-ConvNets-for-regression